

Workshop Python Image Analysis

Martijn Wehrens, May 2026

Estimated time: 120 mins exercises

Chapter 5: Combining building blocks into a pipeline

To recap, remember we have seen:

- Image **thresholding** creates a **mask**
 - This can be **labeled**
 - And **regionprops** can be determined.
- **Region-based processing** to manipulate an image (using labeled image or properties)
- We can also *apply* morphology operations
 - **transformations**
 - **blurring**
 - **edge detection**
 - and more
- We can combine these operations
- We can combine images using numpy operations,
 - **np.logical_and**, np.logical_or, np.isin, etc..
- There are more specialized algorithms that can do similar things
 - Explore documentation/materials at:
 - * Scikit documentation: <https://scikit-image.org/docs/stable/api/skimimage.morphology.html>
 - * Scikit tutorials: https://scikit-image.org/docs/stable/auto_examples/index.html#morphology
 - * Scipy documentation, e.g. for ndimage: <https://docs.scipy.org/doc/scipy/reference/ndimage.html>
- We didn't cover advanced topics, and leave those to avid learners, such as:
 - Fourier transform

- Deconvolution
- ..
- Explore more materials at:
 - <https://bioimagebook.github.io/>
 - <https://haesleinhuepf.github.io/BioImageAnalysisNotebooks/intro.html>

Automation

To automate a pipeline, the following components are useful:

- Loops
- Functions
 - *Try to code in a modular fashion!*
- Interaction with the filesystem using:
 - `os`, `glob.glob`

These topics were covered in our previous “Introduction to Python” course.

Writing reusable, parameterized pipelines

Additionally, it’s good practice to write scripts that can process any dataset, rather than scripts tailored hard-coded for a specific one.

It is convenient to **separate functionality and execution**.

Write **scripts that process your data in a generalized way**, such that they can be applied to any dataset. Parameter settings, data set paths, and other settings specific to specific datasets are not hard coded in the generalized script, but provided as **parameters**.

You can use one of the following strategies to then execute your script.

1. Your generalized script is imported like a “mini library”

After you have a generalized script, you write a separate script that is tailored toward a specific dataset, which imports all functionalities from the main script.

This **tailored script** (`.py` or `.ipynb`) does **contain hard coded parameters**, and is basically your administration of how a dataset-specific run of your pipeline was performed.

You make a script like this for each dataset, or otherwise organize scripts like this in a way to easily find back how your analysis where performed. It might be smart to store these scripts in a **separate (sub)folder** like `/runs/` or `/analyses/`.

2. Your generalized script reads command line arguments, or a config file

Somewhat more advanced.

Write a generalized script, and execute it by **command line**, and have it read its own arguments and **config files**.

To read input arguments, use

- `sys.argv`, or
- `argparse` ([documentation](#))

If you have many tunable parameters, you can store them in a file, e.g.:

- read a table using `csv` (built-in python library)
- read an excel file using `pandas.read_excel()`
- use `yaml` (or `json`) to read config files

3. Other approaches

If you really have a large pipeline you can make use of more advanced approaches like [Snake-make](#), which is a **workflow management system** that allows you to define complex pipelines with dependencies between steps.

Organize your output!

Additionally, it's important to make sure that when you look at your output folders, **you know how the output was generated**. Export a text file (`analysis.log` or something) to your output directory which lists the script file used to generate the output and (importantly) all parameter settings.

Prevent looking at a subpanel in your manuscript figure and thinking: **“Now where did this data come from again, and how am I going to write methods?”**

Guided exercises

Bacterial data

Using [these building blocks](#), design a pipeline such that:

- For all bacterial data, the images get segmented.
 - It might be convenient to store the segmentation masks somewhere, use `np.savez_compressed` or similar.
- Plot the observed bacterial areas over time.

Likely your segmentation won't be perfect. In the manuscript, the algorithm was tweaked and segmentation manually corrected and curated. This is an exercise. Don't try to do a perfect job. Use a segmentation function based on chapter 3, and have that loop over the images.

Nuclear data

Design a pipeline such that:

- For each frame, the nuclei get segmented.
 - It might be convenient to store the segmentation masks somewhere, use `np.savez_compressed` or similar.
- First, for a single image try to do the following:
 - Combine data from the segmentation masks and the other fluorescent channels.
 - * In the 2nd (ie channel 1) fluorescent channel, the KTR signal is recorded.
 - You can open the data in FIJI to check.
 - * Using your nuclear mask based on the channel 0, calculate the mean signal for each respective nuclei.
- Can you now extract the nuclear KTR signals from nuclei for multiple images? And plot this over time?
- Next, use morphology operations to create a mask that corresponds to a small ring of pixels around the nuclei (ie overlapping with cytoplasm, but excluding the nucleus itself).
 - Now extract the signal from those regions as well.
 - Plot this signal too.
- Now for each time t, determine the average nuclear and cytoplasmic signal.
- Plot the nuclear:cytoplasmic ratio over time.
- Can you guess what was done in this experiment?

Background correction

An important consideration when quantifying intensity data, is correcting for the background level.

For example, consider the following example, where intensity values corresponding to measurements on fluorescent signal are shown:

The following table shows the ratio between the nuclear and cytoplasmic signal, both based on the raw signal, and on the background subtracted signal. This is done both for the inactive and active kinase.

	Inactive kinase		Active kinase	
	Raw signal	Background subtracted	Raw signal	Background subtracted
Nuclear signal	145	123	30	8
Cytoplasmic signal	40	18	97	75
Ratio	3.6	6.8	0.3	0.1
Fold-change			12	68

You can see that subtracting the background leads to rather different picture. When you look at the fold-change in ratio, there's even more than a factor 5 difference with or without background correction!

Ways to estimate the background

A common way to correct for the background level is to subtract a single value that represents the average intensity that you attribute to background signal.

Subtracting the mode. In many cases, the mode (most frequently observed value) offers a good estimate for that level. This is applicable when most pixels in your picture are background pixels, and they appear as a large peak in the histogram.

Subtracting mean from known background location. When you know certain parts of your image will never produce any signal (e.g. because there are no cells in a cell culture picture), you can estimate the background by averaging across that region.

There exist multiple more complex algorithms to perform background correction.

For example, the **Rolling ball** algorithm. When you have foreground features of a certain typical size in your image and you have a varying background, you can apply the [rolling ball](#) algorithm. In Python, you can for example use `skimage.restoration.rolling_ball`. [This schematic](#) explains the idea nicely (the algorithm was described first by [Sternberg in 1983](#)).

Flat-field correction

Another type of artifact can occur in quantitative intensity measurements on a microscope when the sample is illuminated unevenly. This can be corrected for by taking a reference image based on a uniform sample, for example a glass slide evenly covered by a dye. (Or better yet, the per-pixel mean over many pictures of that slide.)

Exercise

Can you apply a background correction to the nuclear data, and re-calculate the ratios?

Napari

We've seen that sometimes computer annotations require human correction.

A very convenient tool in image analysis is **Napari**. This allows a user to interactively view and edit images and segmentation masks. It also allows you to write custom functions that can be executed on click, or with widgets.

Let's load some example data:

```
import tifffile as tiff
import numpy as np
import matplotlib.pyplot as plt

array_npz = np.load('images/biological/ecoli_segmentation_result_250.npz')
img_ecoli = array_npz["img_crop"]
img_ecoli_mask = array_npz["segmask"]

fig, axs = plt.subplots(1, 2)
axs[0].imshow(img_ecoli, cmap='grey')
axs[1].imshow(img_ecoli_mask, cmap = 'nipy_spectral', alpha=0.5)
plt.show()
```

Now, it would be nice to be able to correct the mask in the picture above. Napari offers a way to do this:

```
# now show this data in napari
import napari
from skimage.measure import label
from magicgui import magicgui

# -----
```

```

# basic napari initialization

# initialize a napari instance
viewer = napari.Viewer()
# add the image
viewer.add_image(img_ecoli, name='bacteria')
# add the mask as a "label" layer (has specific functionalities)
viewer.add_labels(img_ecoli_mask, name='segmentation')

# -----
# Adding a functionality

# define a small function that re-labels the mask,
# and add it as a button in the napari side panel
@magicgui(call_button="Re-label segmentation")
def relabel_mask():
    # get the current mask from the napari layer
    current_mask = viewer.layers['segmentation'].data
    # re-label it using skimage.measure.label
    new_mask = label(current_mask)
    # update the napari layer with the new mask
    viewer.layers['segmentation'].data = new_mask
# add the button+function as a "widget" to napari
viewer.window.add_dock_widget(relabel_mask, area='right')

# -----
# run napari
napari.run()

# -----
# Accessing data after closing napari window

# after closing napari, access the (possibly re-labeled) mask
corrected_mask = viewer.layers['segmentation'].data

# -----
# Further code
# which can e.g. be saved or displayed
_ = plt.imshow(corrected_mask, cmap = 'nipy_spectral')

```

In the above example we have added a small functionality, that introduces a button on top of the default napari window. (The section **Adding a functionality** can be left out and the

code will still work.)

i Technical note

The @ sign in the `@magicgui(call_button="Re-label segmentation")` syntax, is called a “decorator”. It allows you to add extra functionality to a function in a convenient way. In this case, it adds the functionality of creating a button in the napari side panel that calls the function when clicked. This python syntax is not covered here, see e.g. [here](#) for more information.

Napari is highly customizable, and you can add many custom functions to it. Developers of image analysis packages often use Napari to make it easy for users to interact with pictures and their functionalities.

It’s beyond the scope of this workshop to show all Napari features, but I wanted to make you aware a tool like this exists.

Exercise

Play around with the above code to try out napari.